Programa:	ConcordFTP 3.0	
Protección:	ASProtect 2.0x [Registered] + Trial 30 días	
Objetivos:	evitar que expire	
URL descarga:	http://www.filestream.com	
Dificultad:		
Herramientas:	OllyDbg v1.10 +PEiD 0.94 + ResHacker 3.4	
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS]	Fecha: 13-11-05

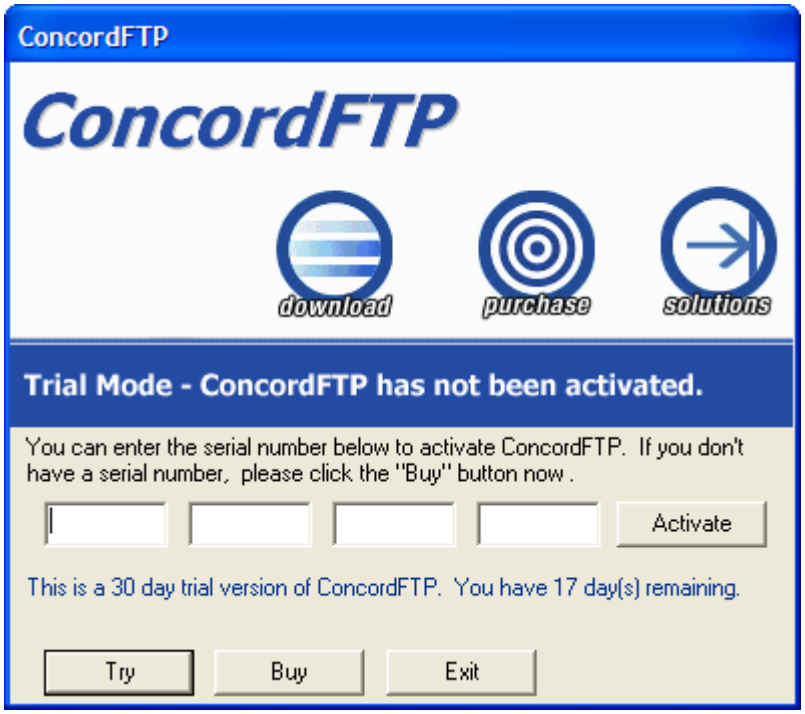
Hola a todos los CracksLatinoS ;-)

Hoy vamos a tratar de registrar el programa "ConcdFTP 3.0" todavía no lo he probado pero por el nombre imagino que servirá para conexiones con FTP's ¿? En fin que ni lo he visto, jeje.. lo que si he visto es que viene con una versión Trial de 30 días + nag incluída en el pack! Un coñazo vaya dobretodo para el que le guste y lo quiera utilizar en su máquina (que no es mi caso xDD)

Lo primero que haremos será ejecutar PEiD y mirar si está empakado y/o encriptado:

Por lo visto no está encriptado ni comprimido, bueno ¿? A veces parece que va a ser fácil y te deja tirado no nos fiaremos, he de decir que aún no he mirado nada de la protección, sólo lo he instalado, antes me gustaba ir escribiendo mientras lo miraba pero dejé este sistema porque algunos programas me dejaban tirado en un punto porque no lograba habilitar cierta función, o lo desempakaba y no lograba que corriera bien, etc., y me quedaban textos de hasta 22 páginas (no os exagero) de Word inútiles que no podía presentaros, por esto mismo dejé de hacerlo así, supongo que me entenderéis ;-)

Bueno que me enrollo y no voy al tema, ejecutemos el programa y somos bien recibidos:

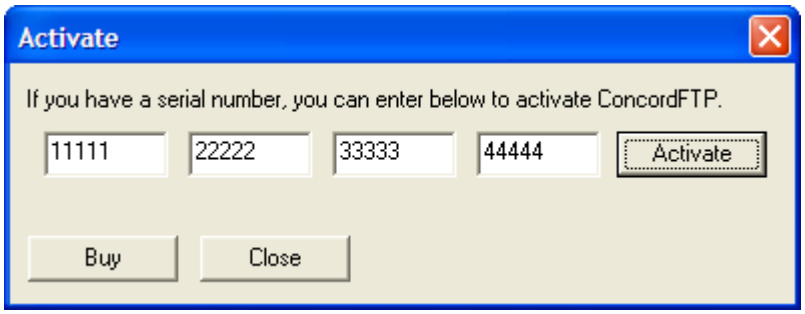


Pedazo de nag exigiendo un código de desbloqueo que por su puesto no conocemos claro, pero esto afortunadamente tiene solución si te has leído todas las teorías y colaboraciones de la lista, jeje.. de momento pulsamos el botón "Try" y el programa arranca, o sea que de antidebug bien poco, la verdad que casi me alegro porque no puedo superar casi ninguno, jeje… tendré que leer más los pedazo de tutes del amigo kaos ;-)

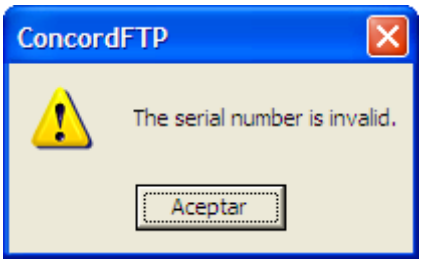
Visto esto ya podemos empezar a planear un atake, yo veo dos puntos claros, uno que me gusta y otro que me gusta más: el que me gusta es atacar la limitación temporal evitando la nagscreen que vemos y el segundo es scarle un serial válido pero esto a veces no es fácil y terminamos parcheando la rutina de verificación, veremos como lo

haremos, en primer lugar vamos a tratar lógicamente de obtener un serial válido, así que nos ponemos en marcha…

Dentro del programa también tenemos la opción de registro en el menú "Help->Activate Now" pulsamos y nos aparece un pequeño cuadro de registro donde ponemos cualquier cosa:



Aceptamos pulsando "Activate" y…



Perfecto..!! típico mensaje de error, parece además un MessageBox por el icono de Exclamación, vamos a Olly y ponemos un BP en la API:

Una vez puesto pulsamos Enter y ya lo tenemos preparado, volvemos a pulsar "Activate" para que vuelva a recoger los datos de registro pero ahora Olly para ;-)

```
77D36476   833D D0C3D677 CMP DWORD PTR DS:[77D6C3D0],0
77D3647D   0F85 885B0100 JNZ USER32.77D4C00B
77D36483   6A 00         PUSH 0
77D36485   FF7424 14     PUSH DWORD PTR SS:[ESP+14]
77D36489   FF7424 14     PUSH DWORD PTR SS:[ESP+14]
77D3648D   FF7424 14     PUSH DWORD PTR SS:[ESP+14]
77D36491   FF7424 14     PUSH DWORD PTR SS:[ESP+14]
77D36495   E8 03000000   CALL USER32.MessageBoxExA
77D3649A   C2 1000       RETN 10
```

Esto forma parte de la API y no nos interesa pulsamos Ctrl+F9 + F8 y estamos en el código del ejecutable, antes aparece el mensaje de error en pantalla que debemos cerrar, pero aún es pronto, vamos retornando y caemos aquí:

```
0054AA65  $  55          PUSH EBP
0054AA66  .  8BEC        MOV EBP,ESP
0054AA68  .  E8 542B0100 CALL concdftp.0055D5C1
0054AA6D  .  8B40 04     MOV EAX,DWORD PTR DS:[EAX+4]
0054AA70  .  85C0        TEST EAX,EAX
0054AA72  .v 74 15       JE SHORT concdftp.0054AA89
0054AA74  .  FF75 10     PUSH DWORD PTR SS:[EBP+10]
0054AA77  .  8B10        MOV EDX,DWORD PTR DS:[EAX]
```

Si miramos en el stack vemos:

```
0012F1D8  0012F200
0012F1DC  0054AAD7  RETURN to concdftp.0054AAD7 from concdftp.0054AA65
0012F1E0  01264E88  ASCII "The serial number is invalid."
0012F1E4  00000000
0012F1E8  0000FBAE
0012F1EC  00000000
0012F1F0  01264E88  ASCII "The serial number is invalid."
0012F1F4  0012F410  Pointer to next SEH record
0012F1F8  00586660  SE handler
```

Parece que ya ha decidido todo, si ya tiene en la pila la cadena de error será porque ya ha verificado el serial, entonces esto estará más para atrás, retornamos un poco más a ver si encontramos el lugar adecuado:

```
004A5C12  .  8D45 F0       LEA EAX,DWORD PTR SS:[EBP-10]
004A5C15  .  50            PUSH EAX                                  pHandle
004A5C16  .  68 BC655F00   PUSH concdftp.005F65BC                    Subkey = "Software\FileStream.c
004A5C1B  .  68 02000080   PUSH 80000002                             hKey = HKEY_LOCAL_MACHINE
004A5C20  .  FF15 0890580  CALL DWORD PTR DS:[<&ADVAPI32.RegCreate  RegCreateKeyA
004A5C26  .  85C0          TEST EAX,EAX
004A5C28  .v 75 4F         JNZ SHORT concdftp.004A5C79
004A5C2A  .  FF35 98B4600  PUSH DWORD PTR DS:[60B498]
004A5C30  .  8D85 FCFDFFF  LEA EAX,DWORD PTR SS:[EBP-204]
004A5C36  .  50            PUSH EAX
004A5C37  .  E8 C4620700   CALL concdftp.0051BF00
004A5C3C  .  8D85 FCFDFFF  LEA EAX,DWORD PTR SS:[EBP-204]
004A5C42  .  50            PUSH EAX
004A5C43  .  E8 28690700   CALL concdftp.0051C570
004A5C48  .  83C4 0C       ADD ESP,0C
004A5C4B  .  50            PUSH EAX                                  BufSize
004A5C4C  .  8D85 FCFDFFF  LEA EAX,DWORD PTR SS:[EBP-204]
004A5C52  .  50            PUSH EAX                                  Buffer
004A5C53  .  6A 01         PUSH 1                                    ValueType = REG_SZ
004A5C55  .  56            PUSH ESI                                  Reserved
004A5C56  .  68 B4655F00   PUSH concdftp.005F65B4                    ValueName = "Number"
004A5C5B  .  FF75 F0       PUSH DWORD PTR SS:[EBP-10]                hKey
004A5C5E  .  FF15 1C90580  CALL DWORD PTR DS:[<&ADVAPI32.RegSetVal   RegSetValueExA
004A5C64  .  FF75 F0       PUSH DWORD PTR SS:[EBP-10]                hKey
004A5C67  .  FF15 3490580  CALL DWORD PTR DS:[<&ADVAPI32.RegCloseK   RegCloseKey
004A5C6D  .v EB 0A         JMP SHORT concdftp.004A5C79
004A5C6F  > 68 AEFB0000    PUSH 0FBAE
004A5C74  .  E8 244E0A00   CALL concdftp.0054AA9D                    ;aqui el mensaje de error...
004A5C79  >  834D FC FF    OR DWORD PTR SS:[EBP-4],FFFFFFFF
004A5C7D  .  8D4D 08       LEA ECX,DWORD PTR SS:[EBP+8]
004A5C80  .  E8 50A30900   CALL concdftp.0053FFD5
004A5C85  .  8B4D F4       MOV ECX,DWORD PTR SS:[EBP-C]
```

Esto ya está mejor, ahora estamos en :4A5C79 y el mensaje lo sacó en el CALL anterior, esto significa que no deberíamos haber llegado hasta aquí porque dentro del CALL ya empuja la cadena de error a la pila, significa que debe haber un salto condicional que nos ponga en :4A5C6F justo debajo del JMP, esa línea roja nos indica precisamente esto, si miramos además la ventana auxiliar vemos:

```
Jump from 004A5BF8
```

El salto está en :4A5BF8 vamos para allí:

```
004A5BE4  .  E8 61A10900   CALL concdftp.0053FD4A
004A5BE9  .  E8 6FE60000   CALL concdftp.004B425D
004A5BEE  .  3935 94B4600  CMP DWORD PTR DS:[60B494],ESI             ;esta es la comparacon...
004A5BF4  .  59            POP ECX
004A5BF5  .  6A FF         PUSH -1
004A5BF7  .  56            PUSH ESI
004A5BF8  .v 75 75         JNZ SHORT concdftp.004A5C6F
004A5BFA  .  68 2BFF0000   PUSH 0FF2B
004A5BFF  .  E8 994E0A00   CALL concdftp.0054AA9D
004A5C04  .  8D45 08       LEA EAX,DWORD PTR SS:[EBP+8]
004A5C07  .  B9 98B46000   MOV ECX,concdftp.0060B498
```

Vemos el salto, es controlado por la comparación que hay entre ESI y [60B494] en :4A5BEE, significa que la variable debe tener el mismo valor que ESI, com ono sabemos lo que valen ponemos un BP en el CALL anterior com ose ve en la imagen y volvemos a pulsar el botón Activate, Olly para en el BP, como no vemos nada raro por ahora hacemo una búsqueda de las referencias a la variable [60B494] esto lo hacemos picando con el botón derecho sobre ella y eligiendo "Find references to->Address constant" sale esto:

References in concdftp:.text to 0060B494

Address	Disassembly	Comment
004A4D96	CMP DWORD PTR DS:[60B494],EBX	
004A5BEE	CMP DWORD PTR DS:[60B494],ESI	(Initial CPU selection)
004A5D16	CMP DWORD PTR DS:[60B494],0	DS:[0060B494]=00000001
004AD8F2	CMP DWORD PTR DS:[60B494],0	DS:[0060B494]=00000001
004B32EE	CMP DWORD PTR DS:[60B494],0	DS:[0060B494]=00000001
004B3304	CMP DWORD PTR DS:[60B494],0	DS:[0060B494]=00000001
004B36F1	CMP DWORD PTR DS:[60B494],0	DS:[0060B494]=00000001
004B3707	CMP DWORD PTR DS:[60B494],0	DS:[0060B494]=00000001
004B4315	AND DWORD PTR DS:[60B494],0	DS:[0060B494]=00000001
004B43A2	MOV DWORD PTR DS:[60B494],EDI	
004B4578	CMP DWORD PTR DS:[60B494],ESI	
004B459E	CMP DWORD PTR DS:[60B494],ESI	
004B4633	CMP DWORD PTR DS:[60B494],EDI	
004B489D	CMP DWORD PTR DS:[60B494],EBX	
004B89B7	CMP DWORD PTR DS:[60B494],0	DS:[0060B494]=00000001

A priori sólo hay una que pone un 0 en la variable y es la que he seleccionado de la lista anterior, si hacemos soble clic en la línea seleccionada de la imagen caeremos en la zona de código en la que está la variable, así podremos echar un vistazo en busca de pistas que nos ayuden a encontrar la solución, veamos donde nos lleva:

```
004B42FE  .   8D85 F0FEFFF LEA EAX,DWORD PTR SS:[EBP-110]
004B4304  .   50           PUSH EAX
004B4305  .   E8 3E8A0600  CALL concdftp.0051CD48          ;Y este CALL?
004B430A  .   83C4 0C      ADD ESP,0C
004B430D  .   3905 9CB4600 CMP DWORD PTR DS:[60B49C],EAX   ;esta comparando..!!
004B4313  .v  75 07        JNZ SHORT concdftp.004B431C
004B4315  .   8325 94B4600 AND DWORD PTR DS:[60B494],0     ;pone la variable a 0..!!
004B431C  >   834D FC FF   OR DWORD PTR SS:[EBP-4],FFFFFFFF
```

Allí caemos, nos llama la atención que está verificando otra variable [60B49C] que compara con el contenido de EAX, pongamos un BP en el CALL, y aparte otro en el inicio de la rutina que es este:

```
004B425D  r$  B8 5C555800  MOV EAX,concdftp.0058555C        ;inicio_rutina..!!
004B4262  .   E8 59700600  CALL concdftp.0051B2C0
004B4267  .   81EC 0802000 SUB ESP,208
004B426D  .   8B45 08      MOV EAX,DWORD PTR SS:[EBP+8]
004B4270  .   8365 FC 00   AND DWORD PTR SS:[EBP-4],0
004B4274  .   8378 F8 00   CMP DWORD PTR DS:[EAX-8],0
004B4278  .v  0F84 9E00000 JE concdftp.004B431C
004B427E  .   FF35 98B4600 PUSH DWORD PTR DS:[60B498]
004B4284  .   50           PUSH EAX                         Arg1
004B4285  .   E8 307F0600  CALL concdftp.0051C1BA           concdftp.0051C1BA
004B428A  .   59           POP ECX
004B428B  .   85C0         TEST EAX,EAX
004B428D  .   59           POP ECX
004B428E  .v  0F84 8800000 JE concdftp.004B431C
004B4294  .   FF75 08      PUSH DWORD PTR SS:[EBP+8]
004B4297  .   8D85 F0FEFFF LEA EAX,DWORD PTR SS:[EBP-110]
```

Allí pondremos otro para estudiar bien lo que hace, ahora si pulsamos Activate por enésima vez ;-) y Olly para en el primer BP que tenemos puesto:

```
004A5BE1  .   8975 FC      MOV DWORD PTR SS:[EBP-4],ESI
004A5BE4  .   E8 61A10900  CALL concdftp.0053FD4A
004A5BE9  .   E8 6FE60000  CALL concdftp.004B425D
004A5BEE  .   3935 94B4600 CMP DWORD PTR DS:[60B494],ESI   ;esta es la comparacon...
004A5BF4  .   59           POP ECX
```

Entramos en el CALL con F7 y aparecemos al inicio de la rutina donde teníamos puesto otro BP por si las moscas, a partir de aquí traceamos con cuidado mirando todo lo que hace, lo primero que nos puede llamar la ateción es este CALL:

```
004B427E  .   FF35 98B4600 PUSH DWORD PTR DS:[60B498]
004B4284  .   50           PUSH EAX                         Arg1
004B4285  .   E8 307F0600  CALL concdftp.0051C1BA           concdftp.0051C1BA
004B428A  .   59           POP ECX
004B428B  .   85C0         TEST EAX,EAX
004B428D  .   59           POP ECX
004B428E  .v  0F84 8800000 JE concdftp.004B431C
```

Si miramos la pila vemos nuestro serial y otro de la misma longitud ¿?

```
0012EFEC  01264E88  Arg1 = 01264E88 ASCII "11111222223333344444"
0012EFF0  01263D08  ASCII "TH3A2EDED454DL5PPPLN"
0012EFF4  77E505FF  RETURN to kernel32.77E505FF from kernel32.77E5A39F
0012EFF8  01264D9D
0012EFFC  001C7620
```

Si vamos rápidamente a probarlo nos damos cuenta que no es válido ¿de una versión anterior? Jeje… podría estar blacklisteado y nos hemos jugado el físico xDD, veamos una curiosidad, miremos donde apunta el salto después del CALL que compara estos seriales:

```
004B4274  .   8378 F8 00   CMP DWORD PTR DS:[EAX-8],0
004B4278  .v  0F84 9E00000 JE concdftp.004B431C             ;ESTE ES EL SALTO..!!
004B427E  .   FF35 98B4600 PUSH DWORD PTR DS:[60B498]
004B4284  .   50           PUSH EAX                         Arg1
004B4285  .   E8 307F0600  CALL concdftp.0051C1BA           concdftp.0051C1BA
004B428A  .   59           POP ECX
004B428B  .   85C0         TEST EAX,EAX
004B428D  .   59           POP ECX
004B428E  .v  0F84 8800000 JE concdftp.004B431C
004B4294  .   FF75 08      PUSH DWORD PTR SS:[EBP+8]
004B4297  .   8D85 F0FEFFF LEA EAX,DWORD PTR SS:[EBP-110]
004B429D  .   50           PUSH EAX
004B429E  .   E8 5D7C0600  CALL concdftp.0051BF00
004B42A3  .   8D85 ECFDFFF LEA EAX,DWORD PTR SS:[EBP-214]
004B42A9  .   68 04010000  PUSH 104
004B42AE  .   50           PUSH EAX
004B42AF  .   8D85 F0FEFFF LEA EAX,DWORD PTR SS:[EBP-110]
004B42B5  .   50           PUSH EAX
004B42B6  .   E8 95490000  CALL concdftp.004B8C50
004B42BB  .   83C4 14      ADD ESP,14
004B42BE  .   8D85 F0FEFFF LEA EAX,DWORD PTR SS:[EBP-110]
004B42C4  .   FF35 9CB4600 PUSH DWORD PTR DS:[60B49C]       <%u> = 376CA633 (929867315.)
004B42CA  .   68 E45B5E00  PUSH concdftp.005E5BE4           Format = "%u"
004B42CF  .   50           PUSH EAX                         s
004B42D0  .   FF15 2099580 CALL DWORD PTR DS:[<&USER32.wsprintfA>]  wsprintfA
004B42D6  .   8D85 F0FEFFF LEA EAX,DWORD PTR SS:[EBP-110]
```

```
004B42DC  .  50            PUSH EAX
004B42DD  .  E8 8E820600   CALL concdftp.0051C570
004B42E2  .  83C4 10       ADD ESP,10
004B42E5  .  83F8 06       CMP EAX,6
004B42E8  .v 7C 32         JL SHORT concdftp.004B431C
004B42EA  .  8D8D ECFDFFF  LEA ECX,DWORD PTR SS:[EBP-214]
004B42F0  .  8D8405 EAFEF  LEA EAX,DWORD PTR SS:[EBP+EAX-116]
004B42F7  .  51            PUSH ECX
004B42F8  .  50            PUSH EAX
004B42F9  .  E8 027C0600   CALL concdftp.0051BF00
004B42FE  .  8D85 F0FEFFF  LEA EAX,DWORD PTR SS:[EBP-110]
004B4304  .  50            PUSH EAX
004B4305  .  E8 3E8A0600   CALL concdftp.0051CD48          ;Y este CALL?
004B430A  .  83C4 0C       ADD ESP,0C
004B430D  .  3905 9CB4600  CMP DWORD PTR DS:[60B49C],EAX   ;esta comparando..!!
004B4313  .v 75 07         JNZ SHORT concdftp.004B431C
004B4315  .  8325 94B4600  AND DWORD PTR DS:[60B494],0     ;pone la variable a 0..!!
004B431C  >  834D FC FF    OR DWORD PTR SS:[EBP-4],FFFFFFFF
004B4320  .  8D4D 08       LEA ECX,DWORD PTR SS:[EBP+8]
004B4323  .  E8 ADBC0800   CALL concdftp.0053FFD5
004B4328  .  8B4D F4       MOV ECX,DWORD PTR SS:[EBP-C]
```

Va a parar justo a :4B431C justo la instrucción siguiente donde se pone la variable que estamos estudiando a 0, sabemos que debe ser 0 precisamente y ese salto lo evita, así que ya sabemos otra cosa: todos los saltos que apunten a :4B431C no nos interesan puesto que queremos pasar por :4B4315 donde se hace [60B494]==0, con este valor indicará que estamos registrados ;-D pero aún es pronto para parchear intentaremos obtener un serial si es posible lo dejamos como segunda opción, introducimos los mismos datos de registro de inicio: 11111-22222-33333-44444 y veamos que pasa… superamos bien el salto de :4B4278 lógico si el serial no es TH3A2EDED454DL5PPPLN que es el MALO, seguimos y llegamos al "CALL wsprintf" que pasa la cadena 376CA633 a formato decimal quedando 929867315, mira si lalongitud es mayor a 6 en este otro CALL:

```
004B42DD  |.  E8 8E820600   CALL concdftp.0051C570
004B42E2  |.  83C4 10       ADD ESP,10
004B42E5  |.  83F8 06       CMP EAX,6
004B42E8  |.  7C 32         JL SHORT concdftp.004B431C
```

después hace un par de operaciones con él dentro de este CALL:

```
004B42F8  .  50            PUSH EAX
004B42F9  .  E8 027C0600   CALL concdftp.0051BF00          ;este nos molesta..!!
004B42FE  .  8D85 F0FEFFF  LEA EAX,DWORD PTR SS:[EBP-110]
004B4304  .  50            PUSH EAX
004B4305  .  E8 3E8A0600   CALL concdftp.0051CD48          ;Y este CALL?
004B430A  .  83C4 0C       ADD ESP,0C
004B430D  .  3905 9CB4600  CMP DWORD PTR DS:[60B49C],EAX   ;esta comparando..!!
004B4313  .v 75 07         JNZ SHORT concdftp.004B431C
004B4315  .  8325 94B4600  AND DWORD PTR DS:[60B494],0     ;pone la variable a 0..!!
004B431C  >  834D FC FF    OR DWORD PTR SS:[EBP-4],FFFFFFFF
```

en el primero de ellos nos hace una jugarreta al borrar un dígito del código con el que habíamos llegado, lo hace en esta línea:

```
0051BFDE  .  C3            RETN
0051BFDF  >  66:8917       MOV WORD PTR DS:[EDI],DX
0051BFE2  .  8B4424 08     MOV EAX,DWORD PTR SS:[ESP+8]
0051BFE6  .  5F            POP EDI
0051BFE7  .  C3            RETN
0051BFE8  >  8817          MOV BYTE PTR DS:[EDI],DL        ;esto no nos ayuda mucho..!!
0051BFEA  .  8B4424 08     MOV EAX,DWORD PTR SS:[ESP+8]
0051BFEE  .  5F            POP EDI
0051BFEF  .  C3            RETN
```

Si miramos la ventana de datos antes de ejecutar la instrucción vemos:

```
Address   Hex dump                                          ASCII
0012F29B  00 68 3A D1 77 8A 09 1B 00 0D 00 00 00 39 32 39  .h:Ðwè.←.....929
0012F2AB  38 36 37 33 31 35 00 33 33 33 33 33 34 34 34 34  867315.333334444
0012F2BB  34 00 F3 12 00 30 85 D2 77 2C F3 12 00 37 3B D1  4.↨↕.0àÒw,↨↕.7;Ð
0012F2CB  77 00 E0 FD 7F 2C F3 12 00 67 3B D1 77 F8 F2 12  w.ó²⌂,↨↕.g;Ðw°=↕
0012F2DB  00 4B 3B D1 77 0D 00 00 00 30 85 D2 77 00 D3 17  .K;Ðw....0àÒw.Ë↨
```

Si ahora la ejecutamos nos quedará esto:

```
Address   Hex dump                                          ASCII
0012F29B  00 68 3A D1 77 8A 09 1B 00 0D 00 00 00 39 32 39  .h:Ðwè.←.....929
0012F2AB  00 36 37 33 31 35 00 33 33 33 33 33 34 34 34 34  .67315.333334444
0012F2BB  34 00 F3 12 00 30 85 D2 77 2C F3 12 00 37 3B D1  4.↨↕.0àÒw,↨↕.7;Ð
0012F2CB  77 00 E0 FD 7F 2C F3 12 00 67 3B D1 77 F8 F2 12  w.ó²⌂,↨↕.g;Ðw°=↕
0012F2DB  00 4B 3B D1 77 0D 00 00 00 30 85 D2 77 00 D3 17  .K;Ðw....0àÒw.Ë↨
```

Vemos que chafó el 8 y nos dejó un bonito 00, eso crea que sea el final de cadena quedando ahora 929 únicamente ya que quedó separada del resto, porqué es un problema? Porque si salimos del CALL y continuamos llegamos al siguiente CALL donde coge esta cadena precisamente y la paa a formato hexadecimal dejando el resultado en EAX, esto lo hace en este CALL:

```
004B4304  |.  50             PUSH EAX
004B4305  |.  E8 3E8A0600    CALL concdftp.0051CD48       ;Y este CALL?
```

Con ese "PUSH EAX" le pasa un puntero a la cadena que ahora es sólo 929 por culpa del CALL anterior que nos chafó un dígito por la cara, si superamos el CALL vemos:

```
Registers (FPU)
EAX 000003A1
ECX 0012F2A8 ASCII "929"
EDX 77E50500 kernel32.77E50500
EBX 00000001
ESP 0012F198
EBP 0012F3B8
ESI 00000000
EDI 0012F9D4
```

Ese EAX==3A1 en principio no nos dice nada verdad? Hagamos una cosa:

```
Command : 3a1                HEX: 3A1 - DEC: 929 - ASCII: ·i
```

Ahora mejor verdad? Es lo que os decía lo ha pasado a formato hexadecimal y queda 3A1, esto es importante porque después llega esta parte:

```
004B4305  .  E8 3E8A0600   CALL concdftp.0051CD48          ;Y este CALL?
004B430A  .  83C4 0C       ADD ESP,0C
004B430D  .  3905 9CB4600  CMP DWORD PTR DS:[60B49C],EAX   ;esta comparando..!!
004B4313  .v 75 07         JNZ SHORT concdftp.004B431C
004B4315  .  8325 94B4600  AND DWORD PTR DS:[60B494],0     ;pone la variable a 0..!!
004B431C  >  834D FC FF    OR DWORD PTR SS:[EBP-4],FFFFFFFF
```

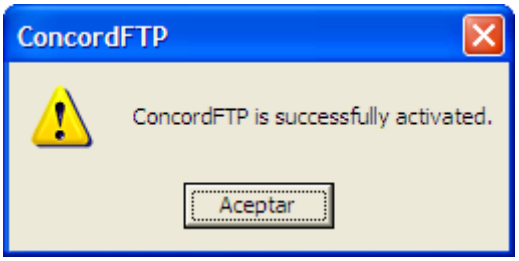
Compara el valor de EAX (el 929 en hexa) con el contenido de la variable [60B49C] si miramos en la ventanita auxiliar veremos su valor:

```
EAX=000003A1
DS:[0060B49C]=376CA633
```

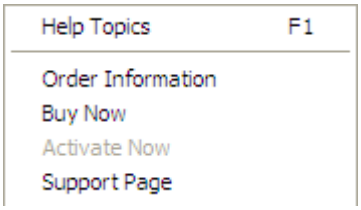
Pues vale justamente 376CA633 que en representación decimal queda 929867315 que es justamente la cadena que teníamos antes de quenos pusiera el carácter 00 separando el 929 del resto, es por eso que dije que nos estaba molestando ahora se entiende supongo, de ser ambos valores iguales no se produciría el salto en :4B4313 y llegaríamos a la instrucción siguiente donde pone la variable [60B494] a 0 que es justamente lo que queremos. Llegado a este punto intenté casi de todo para lograr llegar vivo a :4B4315 pero NADA..!! seguí saltando en :4B4313 cada vez, estudié los flags que hay bastantes y no descubrí mucho pero sí llegué de casualidad al CALL que nos saca la nag inicial:

```
004B48A3  .^ 0F85 02FFFFF  JNZ concdftp.004B47AB
004B48A9  >  391D DCB4600  CMP DWORD PTR DS:[60B4DC],EBX
004B48AF  .v 75 17         JNZ SHORT concdftp.004B48C8
004B48B1  .  3BC3          CMP EAX,EBX
004B48B3  .v 75 13         JNZ SHORT concdftp.004B48C8
004B48B5  .  3BCB          CMP ECX,EBX
004B48B7  .v 75 0F         JNZ SHORT concdftp.004B48C8
004B48B9  .  8BCE          MOV ECX,ESI
004B48BB  .  E8 10FDFFFF   CALL concdftp.004B45D0          ;Nag_Trial y Nag_Expired...
004B48C0  .  85C0          TEST EAX,EAX
004B48C2  .^ 0F84 E3FEFFF  JE concdftp.004B47AB            ;No debe saltar...
004B48C8  >  53            PUSH EBX
004B48C9  .  BF 7C815F00   MOV EDI,concdftp.005F817C       ASCII "Options"
004B48CE  .  68 AC815F00   PUSH concdftp.005F81AC          ASCII "One Instance"
004B48D3  .  57            PUSH EDI
```

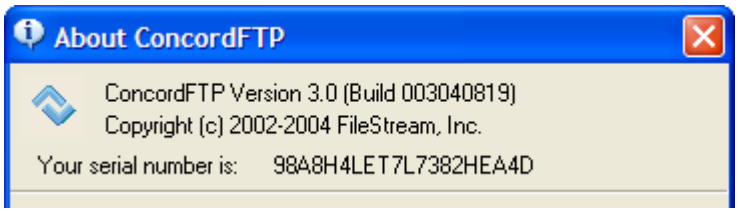
No tiene importancia de la manera que llegué pues todos sabemos poner un BPM en una variable y esperar a que pare, pero como dije antes si no podíamos sacarle un serial válido que es el caso parchearíamos, no quiero terminar sin antes explicar una curiosidad que encontré también: Persiguiendo otro de los flags que tiene descubrí una zona en la que EAX apuntaba a una cadena del tiop "L77L2EDEATAT4558D44E" si contamos los caracteres vemos que son exactamente 20 (los que tiene el serial) pensé que podría ser el válido, lo puse y…



jeje.. pensé al final de rebote hemos pillado un código válido, incluso veremos el menú de esta manera que tiene muy buena pinta:



La opción "Activate Now" desaparece pues queda grisada y no se puede acceder, suponía pues que estaba registrado, includo en el About reconoce el serial como válido:



Allí lo pone bien claro que nuestro serial es 98A8H4LET7L7382HEA4D, todo parecía perfecto pero reinicié y me volvió a aparecer la nag inicial, lo más curioso de todo es que en este CALL (que ya antes vimos):

```
004A5BE9  |.  E8 6FE60000   CALL concdftp.004B425D
004A5BEE  |.  3935 94B46000 CMP DWORD PTR DS:[60B494],ESI       ;aquí compara..
004A5BF4  |.  59            POP ECX
004A5BF5  |.  6A FF         PUSH -1
004A5BF7  |.  56            PUSH ESI
004A5BF8  |.  75 75         JNZ SHORT concdftp.004A5C6F         ;salto MALO..!!
```

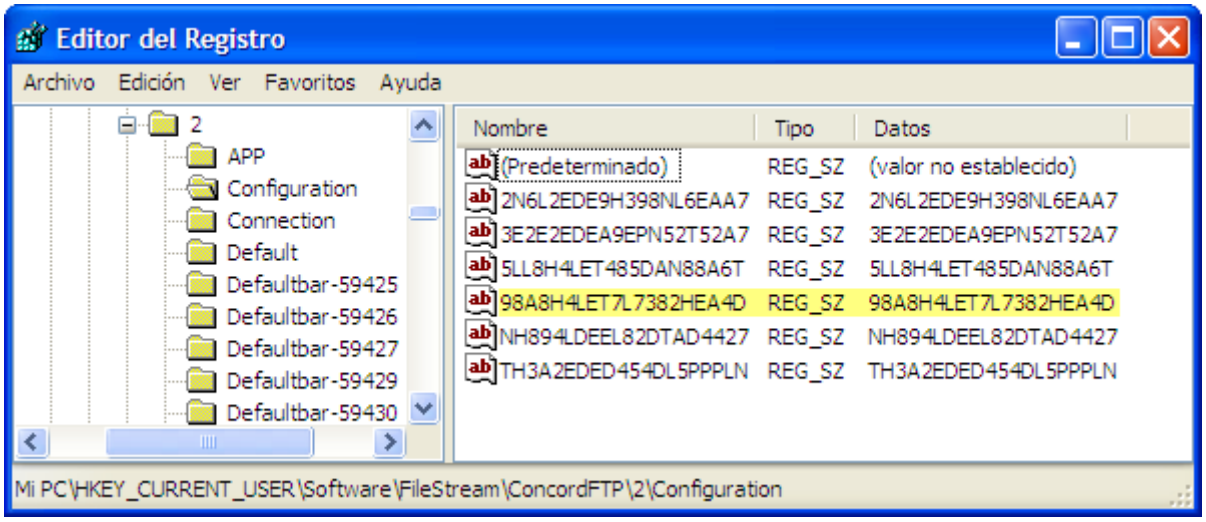
si recordamos miraba si el serial introducido por nosotros era TH3A2EDED454DL5PPPLN, pero ahora si entramos y nos situamos en este subcall:

```
004B4284  |.  50            PUSH EAX                            ; /Arg1
004B4285  |.  E8 307F0600   CALL concdftp.0051C1BA              ; \concdftp.0051C1BA
```

Y miramos en el stack vemos esto:

```
0012E870  01263F38  Arg1 = 01263F38 ASCII "11111222223333344444"
0012E874  01263D08  ASCII "98A8H4LET7L7382HEA4D"
0012E878  77E505FF  RETURN to kernel32.77E505FF from kernel32.77E5A39F
0012E87C  01263E4D
0012E880  00181990
```

Está mirando si el serial es 98A8H4LET7L7382HEA4D justo el que pusimos la última vez y que nos aceptó, investigando descubrí que los tenía en el registro (y algunos más)



Pongamos el que pongamos de esta lista lo aceptará pero no queda registrado ¿? Una pasada vaya, esto lo comento como curiosidad ya que nunca lo había visto antes y ni si quiera sé la finalidad que puede tener hacer esto pero bueno.

Lo que haremos después de darnos mil hostias es parchear directamente, como sabemos donde debemos llegar nos marcamos ese objetivo: Pasar por aquí siempre:

```
004B4315  |.  8325 94B46000>AND DWORD PTR DS:[60B494],0      ;pone la variable a 0..!!
```

Que es donde pone a 0 la variable global que controla si estamos o no registrados, como imagino que esta rutina se llamará ya al inicio del programa para ver si estamos registrados parchearemos aquí mismo, lo que haremos será en principio nopear los 4 saltos que evitan pasar por aquí, los cambios que haremos serán:

```
004B4278   0F84 9E000000 JE concdftp.004B431C         POR:
004B4278   90            NOP
004B4279   90            NOP
004B427A   90            NOP
004B427B   90            NOP
004B427C   90            NOP
004B427D   90            NOP

004B428E   0F84 88000000 JE concdftp.004B431C         POR:
004B428E   90            NOP
004B428F   90            NOP
004B4290   90            NOP
004B4291   90            NOP
004B4292   90            NOP
004B4293   90            NOP

004B42E8   7C 32         JL SHORT concdftp.004B431C   POR:
004B42E8   90            NOP
004B42E9   90            NOP

004B4313   75 07         JNZ SHORT concdftp.004B431C  POR:
004B4313   90            NOP
004B4314   90            NOP
```

Salvamos los cambios con otro nombre tute.exe como siempre, jeje… para qué variar verdad? Dicen que cuando algo va bien no lo cambies pues eso haremos xDD

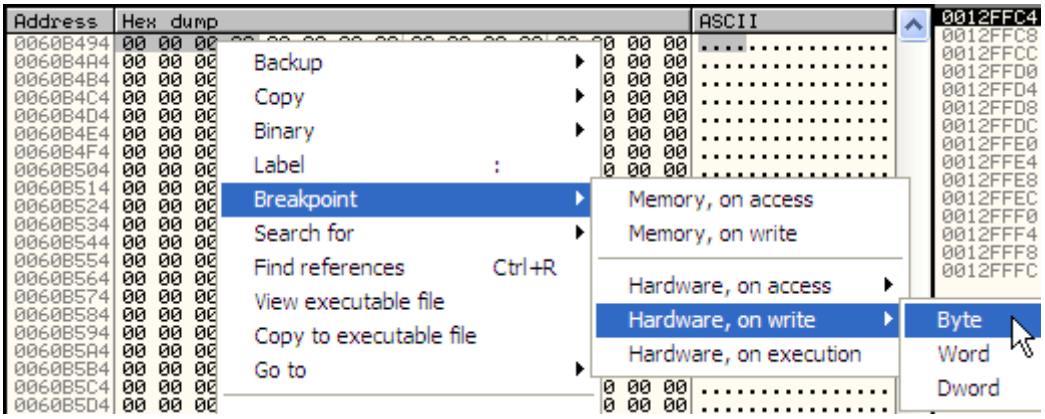
La zona debe quedar así:

```
004B4278     90            NOP                                   ;ESTE ES EL SALTO..!!
004B4279     90            NOP
004B427A     90            NOP
004B427B     90            NOP
004B427C     90            NOP
004B427D     90            NOP
004B427E  .  FF35 98B4600  PUSH DWORD PTR DS:[60B498]
004B4284  .  50            PUSH EAX                              Arg1
004B4285  .  E8 307F0600   CALL concdftp.0051C1BA                concdftp.0051C1BA
004B428A  .  59            POP ECX
004B428B  .  85C0          TEST EAX,EAX
004B428D  .  59            POP ECX
004B428E     90            NOP
004B428F     90            NOP
004B4290     90            NOP
004B4291     90            NOP
004B4292     90            NOP
004B4293     90            NOP
004B4294  .  FF75 08       PUSH DWORD PTR SS:[EBP+8]
004B4297  .  8D85 F0FEFFF  LEA EAX,DWORD PTR SS:[EBP-110]
004B429D  .  50            PUSH EAX
004B429E  .  E8 5D7C0600   CALL concdftp.0051BF00
004B42A3  .  8D85 ECFDFFF  LEA EAX,DWORD PTR SS:[EBP-214]
004B42A9  .  68 04010000   PUSH 104
004B42AE  .  50            PUSH EAX
004B42AF  .  8D85 F0FEFFF  LEA EAX,DWORD PTR SS:[EBP-110]
004B42B5  .  50            PUSH EAX
004B42B6  .  E8 95490000   CALL concdftp.004B8C50
004B42BB  .  83C4 14       ADD ESP,14
004B42BE  .  8D85 F0FEFFF  LEA EAX,DWORD PTR SS:[EBP-110]
004B42C4  .  FF35 9CB4600  PUSH DWORD PTR DS:[60B49C]            <%u> = 376CA633 (929867315.)
004B42CA  .  68 E45B5E00   PUSH concdftp.005E5BE4                Format = "%u"
004B42CF  .  50            PUSH EAX                              s
004B42D0  .  FF15 2099580  CALL DWORD PTR DS:[<&USER32.wsprintfA>]  wsprintfA
004B42D6  .  8D85 F0FEFFF  LEA EAX,DWORD PTR SS:[EBP-110]
004B42DC  .  50            PUSH EAX
004B42DD  .  E8 8E820600   CALL concdftp.0051C570
004B42E2  .  83C4 10       ADD ESP,10
004B42E5  .  83F8 06       CMP EAX,6
004B42E8     90            NOP
004B42E9     90            NOP
004B42EA  .  8D8D ECFDFFF  LEA ECX,DWORD PTR SS:[EBP-214]
004B42F0  .  8D8405 EAFEF  LEA EAX,DWORD PTR SS:[EBP+EAX-116]
004B42F7  .  51            PUSH ECX
004B42F8  .  50            PUSH EAX
004B42F9  .  E8 027C0600   CALL concdftp.0051BF00                ;este nos molesta..!!
004B42FE  .  8D85 F0FEFFF  LEA EAX,DWORD PTR SS:[EBP-110]
004B4304  .  50            PUSH EAX
004B4305  .  E8 3E8A0600   CALL concdftp.0051CD48                ;Y este CALL?
004B430A  .  83C4 0C       ADD ESP,0C
004B430D  .  3905 9CB4600  CMP DWORD PTR DS:[60B49C],EAX         ;esta comparando..!!
004B4313     90            NOP
004B4314     90            NOP
004B4315  .  8325 94B4600  AND DWORD PTR DS:[60B494],0           ;pone la variable a 0..!!
004B431C  >  834D FC FF    OR DWORD PTR SS:[EBP-4],FFFFFFFF
```

Salvamos los cambios con el "Save File" de siempre y probamos el resultado… pues nada no funciona del todo, nos acepta el serial pero no quedamos registrados, necesitamos ser más fuertes en el parcheo, volvamos al lugar donde aparece la nag:

```
004B4873  >  8BCE          MOV ECX,ESI
004B4875  .  E8 0FFBFFFF   CALL concdftp.004B4389
004B487A  .  85C0          TEST EAX,EAX
004B487C  .^ 0F84 29FFFFFF JE concdftp.004B47AB
004B4882  .  A1 6CB36000   MOV EAX,DWORD PTR DS:[60B36C]   <-
004B4887  .  8B0D B8B7600  MOV ECX,DWORD PTR DS:[60B7B8]   <-   Ojo con estas variables..!!
004B488D  .  3BC3          CMP EAX,EBX
004B488F  .v 75 04         JNZ SHORT concdftp.004B4895
004B4891  .  3BCB          CMP ECX,EBX
004B4893  .v 74 14         JE SHORT concdftp.004B48A9
004B4895  >  391D 88B7600  CMP DWORD PTR DS:[60B788],EBX
004B489B  .v 75 0C         JNZ SHORT concdftp.004B48A9
004B489D  .  391D 94B4600  CMP DWORD PTR DS:[60B494],EBX
004B48A3  .^ 0F85 02FFFFFF JNZ concdftp.004B47AB            ;nos saca del programa...
004B48A9  >  391D DCB4600  CMP DWORD PTR DS:[60B4DC],EBX    ;es EBX==0...
004B48AF  .v 75 17         JNZ SHORT concdftp.004B48C8      ;salto_bueno..!!
004B48B1  .  3BC3          CMP EAX,EBX
004B48B3  .v 75 13         JNZ SHORT concdftp.004B48C8      ;salto_bueno..!!
004B48B5  .  3BCB          CMP ECX,EBX
004B48B7  .v 75 0F         JNZ SHORT concdftp.004B48C8      ;salto_bueno..!!
004B48B9  .  8BCE          MOV ECX,ESI
004B48BB  .  E8 10FDFFFF   CALL concdftp.004B45D0           ;call_nag..!!   <-   Ojo..!!
004B48C0  .  85C0          TEST EAX,EAX
004B48C2  .^ 0F84 E3FEFFFF JE concdftp.004B47AB             ;nos saca del programa...
004B48C8  >  53            PUSH EBX
```

Resalto lo que creo más importante, entran en juego un par de flags (variables globales, a veces se les llama así xDD) en concreto [60B36C] y [60B7B8], además tenemos que la que controla el registro [60B494] debe estar a 0, comenzamos con esta última veamos donde se inicializa:



Veamos donde para, vemos que al inicio del programa en el EntryPoint está a 0 así que en algún lugar se le debe pasar ese 1 que no nos gusta, pulsamos F9 y Olly para:

```
004B439C  .  6A 01         PUSH 1
004B439E  .  5F            POP EDI
004B439F  .  894D EC       MOV DWORD PTR SS:[EBP-14],ECX
004B43A2  .  893D 94B4600  MOV DWORD PTR DS:[60B494],EDI   <-   Pone el flag a 1..!!
004B43A8  .  E8 B3FFFFFF   CALL concdftp.004B4360
004B43AD  .  33F6          XOR ESI,ESI
004B43AF  .  A3 9CB46000   MOV DWORD PTR DS:[60B49C],EAX
```

Vemos que ya escribió el 1 allí:

```
Address   Hex dump                                          ASCII
0060B494  01 00 00 00 40 02 60 00 00 00 00 00 40 02 60 00  0...@`.....@`.
0060B4A4  40 02 60 00 40 02 60 00 00 00 00 00 00 00 00 00  @`.@`.........
0060B4B4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0060B4C4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

El registro EDI es el que lleva el valor malo que acaba de coger con el "PUSH 1" y seguido el "POP EDI" así mete un 1 en EDI que luego pasa a nuestra variable, esto lo reparamos bien fácil cambiando:

```
004B439C   6A 01      PUSH 1       POR:
004B439C   6A 00      PUSH 0
```

Así recogerá un 1 cada vez que pase pora aquí, jeje… pero aún no hemos terminado porque el programa continúz expirando, vemos la nag, etc… justo encima del call_nag evíamos como verificaba dos variables más, jugando con ellas descubrí que dejando [60B36C] igual, o sea puesta a 0 y cambiando el valor de [60B7B8] por un 1 el programa corre bien y no expira, tampoco vemos la nag y nos reconoce el sreial com oválido en el About, el único pero es en la barra de tituloque continúa diciendo (Trial Version), no podemos tocar el contenido de la variable [60B36C] porque entonces no arranca, lo que haremos será lo siguiente, dejamos el cambio anterior de cambiar el PUSH para que la

variable de registro esté puesta a 0, después para poner un 1 en [60B7B8], pero tampoco funciona, resulta que la que verdaderamente nos interesa está aquí:

```
004B48A9   391D DCB46000   CMP DWORD PTR DS:[60B4DC],EBX
004B48AF   75 17           JNZ SHORT concdftp.004B48C8
```

Es [60B4DC] la que debe contener un 1 y no la otra me confundí, poniendo un BPM vemos quien la inicializa como hicimos antes con la otra variable, es aquí:

```
004B46EB  .  68 94855F00  PUSH concdftp.005F8594          ASCII "FileStream"
004B46F0  .  8BCE         MOV ECX,ESI
004B46F2  .  A3 DCB46000  MOV DWORD PTR DS:[60B4DC],EAX   <-- Inicializa la variable..
004B46F7  .  E8 AF9B0A00  CALL concdftp.0055E2AB
004B46FC  .  FFB6 9000000 PUSH DWORD PTR DS:[ESI+90]
004B4702  .  E8 4E8B0600  CALL concdftp.0051D255
004B4707  .  8D45 E4      LEA EAX,DWORD PTR SS:[EBP-1C]
004B470A  .  C70424 00E00 MOV DWORD PTR SS:[ESP],0E000
```

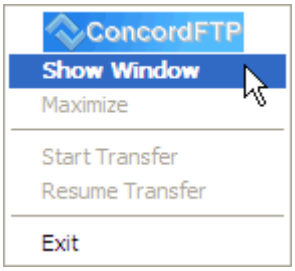
Aquí toma el valor de EAX que es 0, necesitamos que sea 1, miramos por encima y vemos:

```
004B46D8   .  E8 73990600   CALL concdftp.0051E050
004B46DD   .  F7D8          NEG EAX
004B46DF   .  59            POP ECX
004B46E0   .  8935 D8B46000 MOV DWORD PTR DS:[60B4D8],ESI
004B46E6   .  1BC0          SBB EAX,EAX
004B46E8   .  59            POP ECX
004B46E9   .  F7D8          NEG EAX
```

Al final ese "NEG EAX" es el que nos echa todo al traste, para que quede EAX==1 modificamos estas instrucciones de este modo:

```
004B46E6   .  1BC0          SBB EAX,EAX
004B46E8   .  59            POP ECX
004B46E9   .  F7D8          NEG EAX

POR:
004B46E6      33C0          XOR EAX,EAX
004B46E8   .  59            POP ECX
004B46E9      40            INC EAX
004B46EA      90            NOP
```

Salvamos todos los cambios (recordad el PUSH 0 de antes) y lo llamamos tute.exe, el anterior tute.exe no lo borramos que nos servirá ya que tiene el CALL parcheado, este cambio también lo necesitaremos por si las moscas lo dejamos, una vez guardado el exe modificado lo ejecutamos, arranca perfecto e incluso desapareció el botón "Buy Now" que había debajo del menú junto al de ayuda "Help" vamos muuuuuuuy bien, además no vemos la nag, reconoce el serial como válido en el About y no expira ¿se puede pedir alo más? Pues la verdad es que sí: quitar ese horrible (Trial Version) de la barra de título, veamos como lo podemos hacer, pero antes resolvamos otro problema, jeje… esto no acabará nunca, si vamos al menú "File->Close App" y picamos el programa queda minimizado en el tray (bara de tareas) si ahora intentamos abrirlo:



Nos encontramos una desagradable sorpresa: la nag vuelve a aparecer y en Modo_Expired y no nos deja pasar más, jejeje… la que nos tenía guardada, veamos como arreglar esto, reiniciamos Olly con Ctrl+F2 como siempre y lanzamos la aplicación con normalidad, o sea pulsando F9, cuando esté corriendo ya en pantalla vamos al menú "File->Close App" y pulsamos, la aplicación quedará minimizada, ahora intentamos abrirla desde el menú que se ve en la imagen anterior que aparece picando con el botón derecho y eligiendo "Show Window" como la Api, jeje.. entonces nos sacará la nag, peeeeeeero lo vamos a estar esperando ya de antes para pilarlo a él y no él a nosotros ;-)

Recordemos donde mostraba la nag:

```
004B48A3  .^ 0F85 02FFFFFF JNZ concdftp.004B47AB
004B48A9  > 391D DCB4600 CMP DWORD PTR DS:[60B4DC],EBX
004B48AF  .v 75 17         JNZ SHORT concdftp.004B48C8
004B48B1  . 3BC3           CMP EAX,EBX
004B48B3  .v 75 13         JNZ SHORT concdftp.004B48C8
004B48B5  . 3BCB           CMP ECX,EBX
004B48B7  .v 75 0F         JNZ SHORT concdftp.004B48C8
004B48B9  . 8BCE           MOV ECX,ESI
004B48BB  . E8 10FDFFFF    CALL concdftp.004B45D0          ;Nag_Trial y Nag_Expired...
004B48C0  . 85C0           TEST EAX,EAX
004B48C2  .^ 0F84 E3FEFFFF JE concdftp.004B47AB            ;No debe saltar...
004B48C8  > 53             PUSH EBX
004B48C9  . BF 7C815F00    MOV EDI,concdftp.005F817C       ASCII "Options"
004B48CE  . 68 AC815F00    PUSH concdftp.005F81AC          ASCII "One Instance"
004B48D3  . 57             PUSH EDI
```

Era en :4B48BB pero esta llamada la hemos anulado así que debe de llamarla desde otro punto, lo que haremos será entrar dentro y poner un BP al inicio de todo:

```
004B45D0  $ B8 A2555800    MOV EAX,concdftp.005855A2
004B45D5  . E8 E66C0600    CALL concdftp.0051B2C0
004B45DA  . 81EC F805000 SUB ESP,5F8
004B45E0  . 56             PUSH ESI
004B45E1  . 57             PUSH EDI
004B45E2  . 33FF           XOR EDI,EDI
```

Y ahora sí pulsamos en "Show Window" Olly para ;-) vamos traceando hasta llegar al RET final de la rutina, antes nos aparece la nag en pantalla:

Ya no nos deja pasar más, el botón "Try Now" ha desaparecido y en su lugar encontramos "Exit" o sea a la calle, jeje… no pasa nada, si nos fijamos justo antes del CALL vemos:

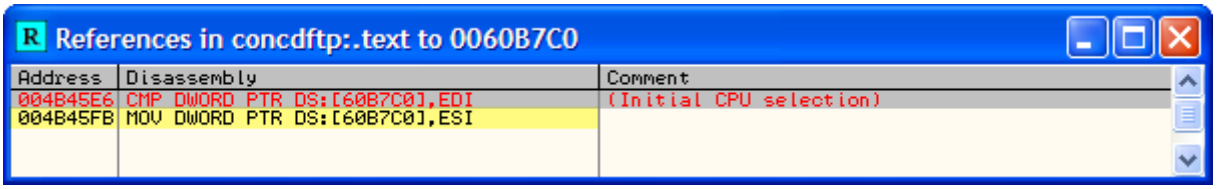
```
004B45E1  . 57             PUSH EDI                        ;salva EDI en la pila
004B45E2  . 33FF           XOR EDI,EDI                     ;hace EDI==0
004B45E4  . 6A 01          PUSH 1
004B45E6  . 393D C0B7600 CMP DWORD PTR DS:[60B7C0],EDI     ;ojo a esta comparacion..!!
004B45EC  .v 74 06         JE SHORT concdftp.004B45F4      ;no deberia saltar
004B45EE  . 58             POP EAX
004B45EF  .v E9 9A000000   JMP concdftp.004B468E
004B45F4  > 393D 88B7600 CMP DWORD PTR DS:[60B788],EDI
004B45FA  . 5E             POP ESI
```

Vemos que pone EDI a 0 para más tarde compararlo con el valor de [60B8C0] hagamos un "Follow in dump->Memory address" a ver cuanto vale:

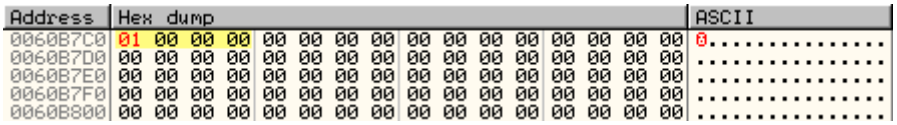
```
Address   Hex dump                                          ASCII
0060B7C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0060B7D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0060B7E0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0060B7F0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0060B800  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Está a 0 entonces saltará en :4B45EC llegando a las instrucciones que sacan la nag en pantalla, es obvio que si esta variable estuviese a 1 no saltaría y pondría un 1 en EAX al hacer el POP ya que antes empujó un 1 a la pila en :4B45E4 saliendo del CALL con EAX==1, pues venga vamos a hacerlo bien, pongamos un BPM en [60B7C0] a ver donde se inicializa a ver si le podemos meter un 1 allí mismo xDDD

Hacemos esto pero no vemos que pare ¿? Si buscamos referencias encontramos sólo una:



Y está un poco más debajo de donde estamos, luego no nos interesa demasiado, probemos a reiniciar Olly y ponerle un 1 así de entrada como no escribe en ella nos puede servir:

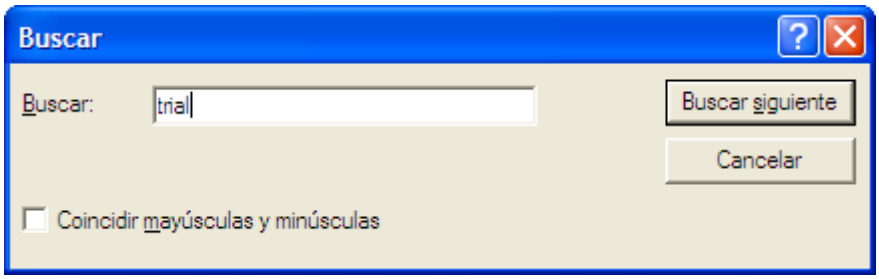


Cuando tenemos el 1 puesto damos RUN y funciona, probamos la función y va bien, pero como no nos soluciona el Trial no sreá muy importante así queno injertaremos para nada, lo que haremos será cambiar:

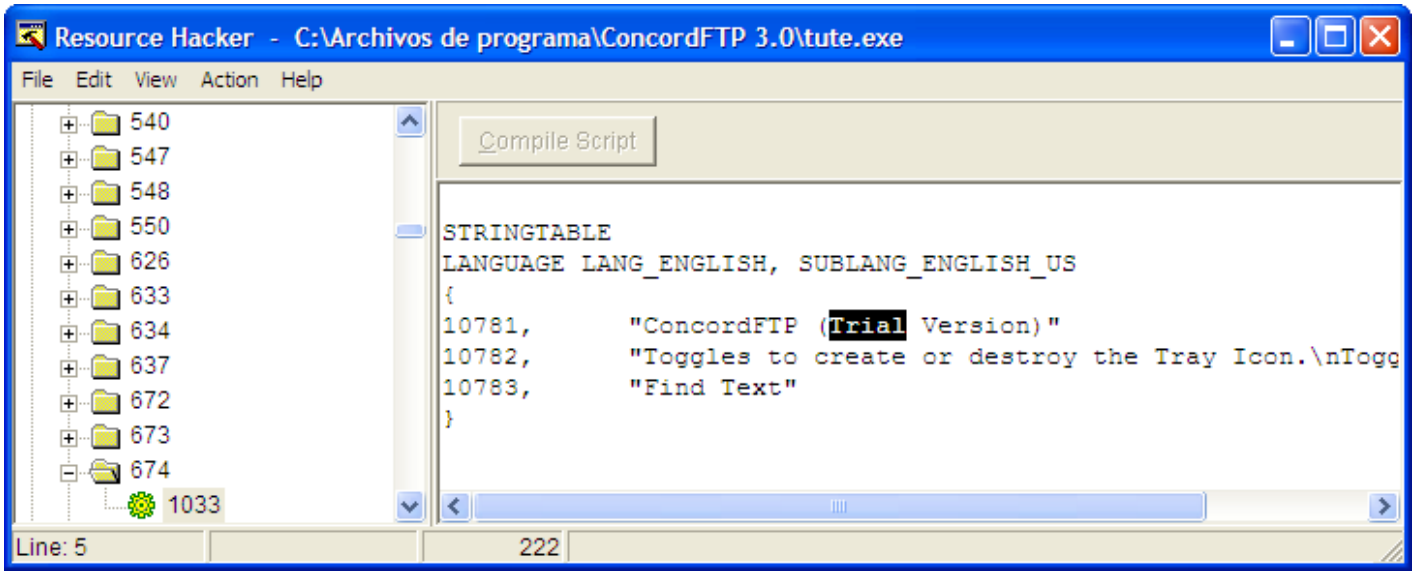
```
004B45E6  393D C0B76000    CMP DWORD PTR DS:[60B7C0],EDI       ; ojo con esto..!!
004B45EC  74 06            JE SHORT concdftp.004B45F4          ; no deberia saltar..!!

POR:
004B45E6  C605 C0B76000 01 MOV BYTE PTR DS:[60B7C0],1          ; lo ponemos a 1..!!
004B45ED  90               NOP
```

Y ya sólo nos queda quitar el (Trial Version) si no hay más sorpresas claro xDD debemos llevar la cuenta de las modificaciones que vamos haciendo para no perdernos, de momento son cuatro y aún no hemos terminado, para estos casos de eliminar una cadena, etc., hay una fantástica herramienta llamada ResHacker (Resource Hacker) que va de maravilla, yo utilizo la versión 3.4 aunque creo que ya la han actualizado, veamos como hacerlo…

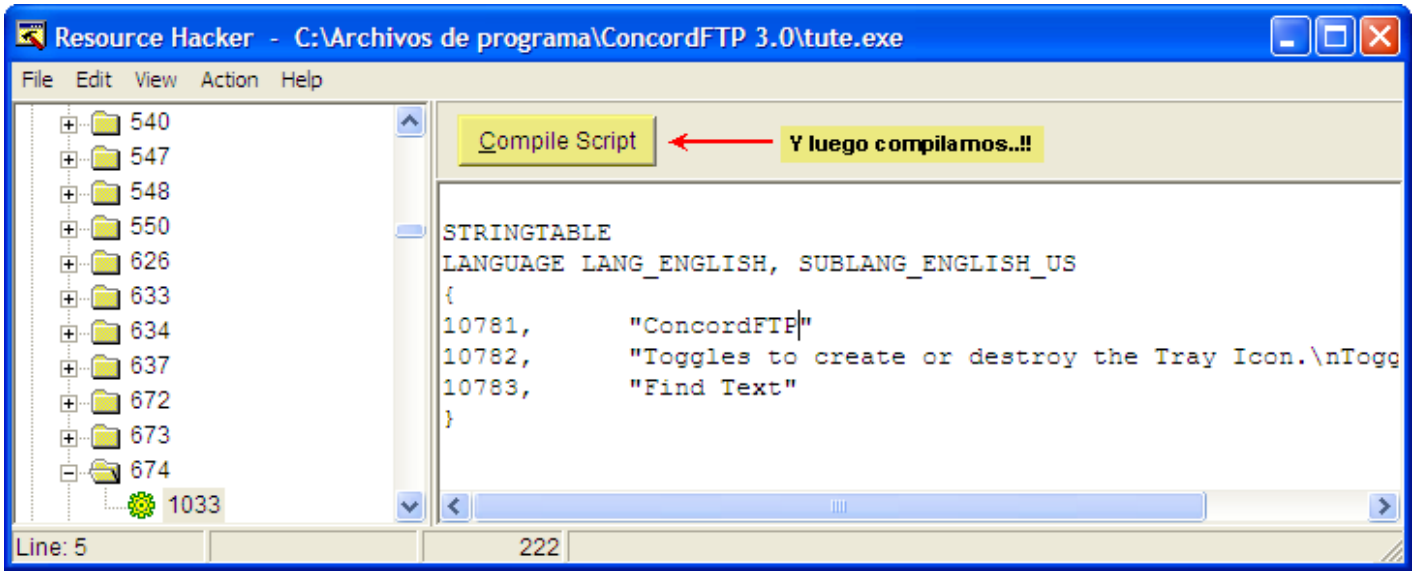
Ejecutamos el ResHacker y abrimos tute.exe que es donde están todas las modificaciones p(las 4 que tenemos) cuando lo tengamos abierto ya pulsamos Ctrl+F y se nos abre un pequeño cuadro donde ponemos la cadena a buscar:



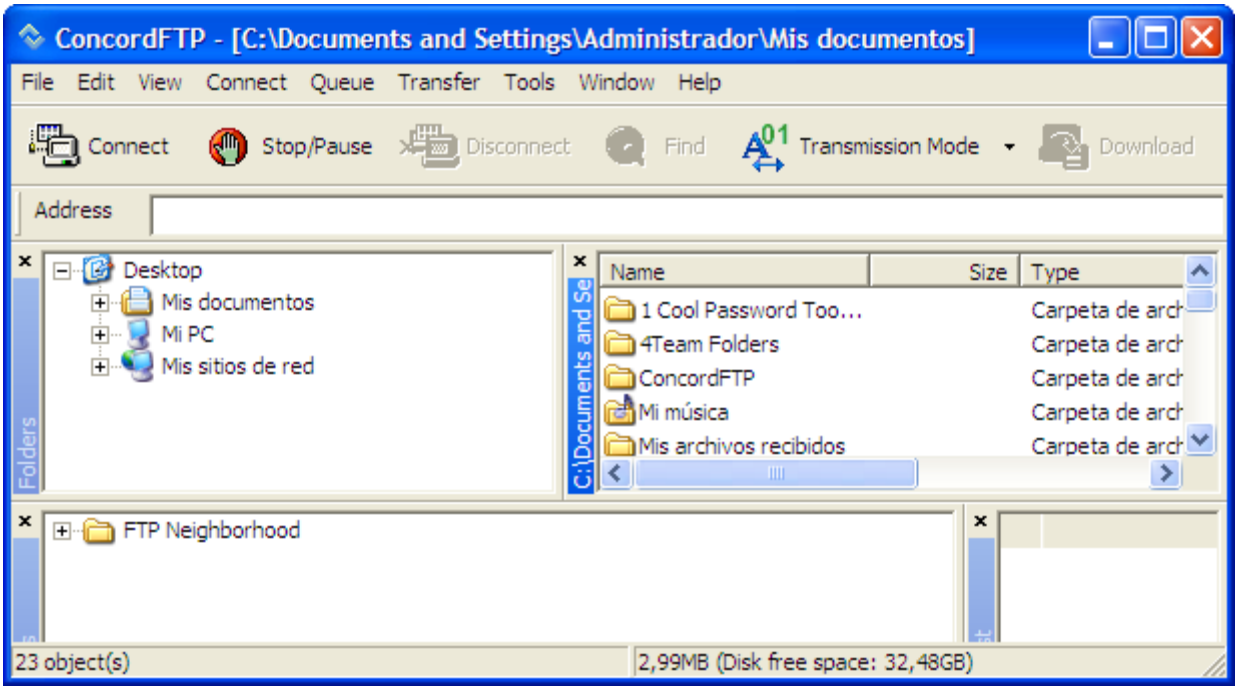
Aceptamos y nos muestra la primera referencia a esta cadena:



Ahí la tenemos, probamos a eliminarla:



Cuando la hayamos borrado compilamos de nuevo con "Compile Script" y después salvamos los cambios con Ctrl+S, ejecutamos tute.exe y…



jejeje… ya no está el (Trial Version) nos lo hemos cargado xDD

Con esta última modificación temina el tute, espero que os haya entretenido tanto como a mi el escribirlo, es un placer, queda dedicado para todo CracksLatinoS sin Escepción alguna, para mi todos son importantes y cumplen su papel, suerte a todos y hasta la próxima que espero que sea pronto ;-))

Para cualquier cosa ya sabéis donde encontrarme:

syxe00@yahoo.es

cualquier email será respondido..!!